developer.nvidia.com

Improve AI Code Generation Using NVIDIA NeMo Agent Toolkit

Christian Munley

16–20 minutes

With the release of NVIDIA NeMo Agent toolkit—an open-source library for connecting and optimizing teams of AI agents—developers, professionals, and researchers can create their own agentic AI applications. This tutorial shows you how to develop apps in the Agent toolkit through an example of AI code generation. We build a test-driven coding agent using LangGraph and reasoning models to scale test-time computation.

Scaling laws are driving smarter AI systems in pre-training, post-training, and inference. The large-scale pretraining of large language models (LLMs) delivers impressive results but is challenging to scale further. Autonomous AI agents and test-time compute methods, such as those used by DeepSeek-R1, are providing notable improvements by scaling post-training and inference compute. This becomes imperative when building agentic workflows for complex tasks such as logic, math, or coding.

These novel scaling methods are simpler to adopt with Agent toolkit, as organizations can better design, test, deploy, and optimize their AI agent applications. Let's dive into how you can improve AI code generation workflows within Agent toolkit.

Why build coding agents with Agent toolkit

LLMs excel at coding tasks but are limited to a chat interface, lacking autonomy and integration with the real world. In contrast, AI agents, powered by these LLMs, are designed to accomplish real-world goals. They often interact with their environment using tools, memory, and planning to execute tasks such as file editing, code execution, or information search.

AI agent design considerations

AI agents are one example of scaling inference-time computation for improving AI performance. To build an agent or multi-agent system, you must balance flexibility against structure.

A flexible agent might be given a shell, a code editor, and a web browser, and be tasked with minimal instruction. In contrast, a structured agent might consist of predefined steps, such as localizing a failed test case within a larger codebase and then executing code changes until the error is resolved. A popular middle ground is flow engineering, where states and transitions are defined, and an agent or tool executes within each state.

Reasoning models and search methods are another example where inference-time computation matters. Reasoning models such as DeepSeek-r1 or OpenAI o1 spend extra time exploring various reasoning paths and solutions within a single chain of thought before providing a final output. Search methods, such as beam search, also explore various branches, leveraging a scoring function such as a verifiable outcome or an approximation.

Ease of AI agent development with Agent toolkit

Evaluation, deployment, and optimization are a few common challenges developers can resolve with Agent toolkit. The following table summarizes some of the features and benefits of Agent toolkit.

Feature	Benefit
Inclusive of agent framework ecosystem	Continue building with your favorite tools like LangGraph and CrewAI.
Common specification	Enables reusability and compatibility across projects, including many examples within Agent toolkit. Projects can be shared through the Agent toolkit registry system.
Evaluation harness	Rapid development and iteration on workflows. Define a set of expected outputs and easily test different models, tools, and workflows by updating the configuration file.
Built-in deployment options	Easily launch microservices with `aiq serve` or leverage the open-source chatbot-style user interface.
Optimization features	Identify bottlenecks with the workflow profiler and leverage features like parallel tool calling and integration with NVIDIA Dynamo for best performance.
Observability	Monitor and debug with tight integration with Phoenix, OpenTelemetry Collector, and custom providers.

Table 1. Features and benefits of Agent toolkit

Please refer to the documentation or GitHub for a detailed list of features.

Tutorial prerequisites

You need the following setup:

- NVIDIA GPUs to run reasoning NIM microservices
- NVIDIA Agent toolkit
- LangGraph framework

How to build an AI code generation agent in NVIDIA Agent toolkit

In this post, we guide you through integrating AI agents and reasoning models to create an AI code-generation agent in Agent toolkit. We build the core agent using LangGraph, integrate a sandbox code execution tool for safety and control, and enhance error correction with DeepSeek-r1. Lastly, we show how the agent can be integrated into a larger system using a supervisor agent.

Set up the project scaffold

First, clone the NVIDIA NeMo Agent toolkit GitHub repository. Follow the instructions in the README to install the Agent toolkit library.

Now create a new project template using the AIQ scaffold command. The scaffold includes a default workflow and configuration file.

```
aiq workflow create code_gen_example
```

NVIDIA NeMo Agent toolkit unifies the concepts of agentic workflows and callable tools under a single class, the function. You can implement the code generation agent as a function, and use it as a callable tool within a supervisor agent, such as a ReACT agent. Other agents, such as a research agent, error localization agent, or test generation agent, can be managed by the supervisor and launched asynchronously for handling complex tasks.

The input to the code generation agent is a problem statement, code to fix, and unit tests. The agent

follows a simple process:

1. Given the problem statement (for example, a GitHub issue), code to fix, and unit tests, the agent uses a code LLM for code generation to create a git patch that resolves the issue.
2. The updated code runs against the unit tests in a safe code execution sandbox.
3. If the test fails, a reasoning model will suggest changes based on the output.
4. Steps 1-3 repeat until either the generated code passes the desired unit tests, or the maximum number of iterations is exceeded.

Update the configuration file

The configuration file in Agent toolkit defines the entire workflow. By updating the configuration file, such as adding tools (functions), swapping LLMs, or changing other components, agentic workflows can be rapidly iterated on with evaluations through the `aiq eval` CLI command.

The scaffold command creates a default config file. You update three sections: `functions`, `llms`, and `workflow`. The `functions` section contains tools accessible to agents, the `llms` section defines which models are available to agents and tools, and the `workflow` section is the main entry point. Here, specify the workflow type as `react_agent`, which uses the default ReACT agent inside the Agent toolkit.

```
functions:
  code_gen_tool:
    _type: code_gen_tool
    debug_llm: reasoning_llm
```

```
    code_llm: code_generation_llm
    max_iterations: 3
llms:
  reasoning_llm:
    _type: nim
    model_name: deepseek-ai/deepseek-r1
    max_tokens: 8000
  code_generation_llm:
    _type: nim
    model_name: qwen/qwen2.5-coder-32b-instruct
    max_tokens: 2048
  general_llm:
    _type: nim
    model_name: meta/llama-3.3-70b-instruct
    max_tokens: 2048
workflow:
  _type: react_agent
  tool_names:
```

```
    - code_gen_tool
  llm_name: general_llm
  verbose: true
  retry_parsing_errors: true
```

In this example, all three LLMs are served with NVIDIA NIM, which can be accessed through the NVIDIA API Catalog or hosted locally. OpenAI and other LLM providers are also supported. For more information, see the NVIDIA NeMo Agent toolkit documentation.

Implement the code generation function

Create the code generation function referenced in the configuration file. In the project scaffold, open the `register.py` file and add the following:

```
class CodeGenToolConfig(FunctionBaseConfig, name="code_gen_tool"):
    reasoning_llm: str
    code_llm: str
    max_iterations: int = 5
@register_function(config_type=CodeGenToolConfig)
async def code_generation(config: CodeGenToolConfig, builder: Builder):
```

Within this function, you define helper functions and a primary runnable function, `_code_gen_tool`, to run when the tool is called. Implement a LangGraph workflow with four steps:

1. The user (or another agent) inputs a problem statement (for example, a GitHub issue), code to fix, and

unit tests that should pass or be fixed. The agent is prompted to create a git patch that resolves the issue, using the configured coding LLM.

2. The updated code runs in a code execution tool to evaluate the results.
3. If the test fails, the reasoning model is prompted to suggest changes based on the problem statement, code, and test output.
4. Steps 1-3 repeat until either the generated code passes the desired unit tests, or the maximum number of iterations is exceeded.

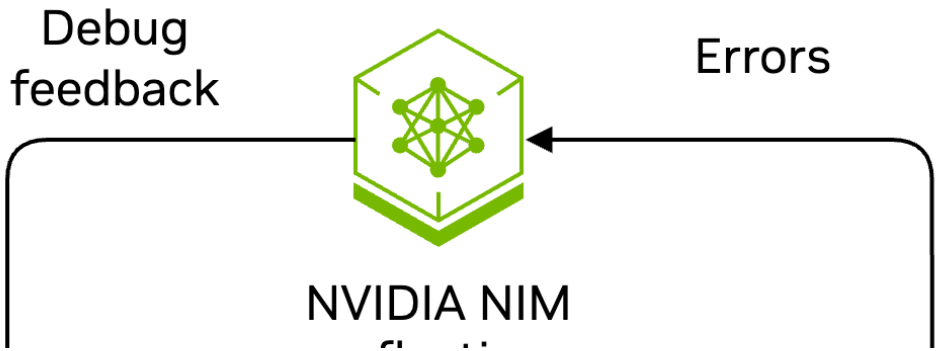
```
workflow = StateGraph(CodeState)
workflow.add_node("code_generation", generate_code)
workflow.add_node("run_unit_test", test_code)
workflow.add_node("debug", debug_code)
workflow.add_edge(START, "code_generation")
workflow.add_edge("code_generation", "run_unit_test")
workflow.add_conditional_edges(
    "run_unit_test",
    should_continue,
    {
        "end": END,
        "debug": "debug"
```

```
    }
)
workflow.add_edge("debug", "code_generation")
agent =workflow.compile()
```

Each node in the LangGraph agent is defined in a Python function, which can be an autonomous agent, a tool call, or anything else. The `generate_code` node uses the Qwen NIM microservice to generate code, the `run_unit_test` node runs the tests against the updated code in a sandbox environment, and the `debug` node uses DeepSeek-R1 for advanced reasoning about failures.

Agent toolkit uses `yield` to register a function as callable from any other function. Providing a detailed and accurate description for functions is critical to developing agents that interact with each other effectively.

```
yield FunctionInfo.from_fn(
    _code_generation,
       description=("This tool is a code generation agent using test driven development. Provide input including the issue, current code, and unit tests."))
```



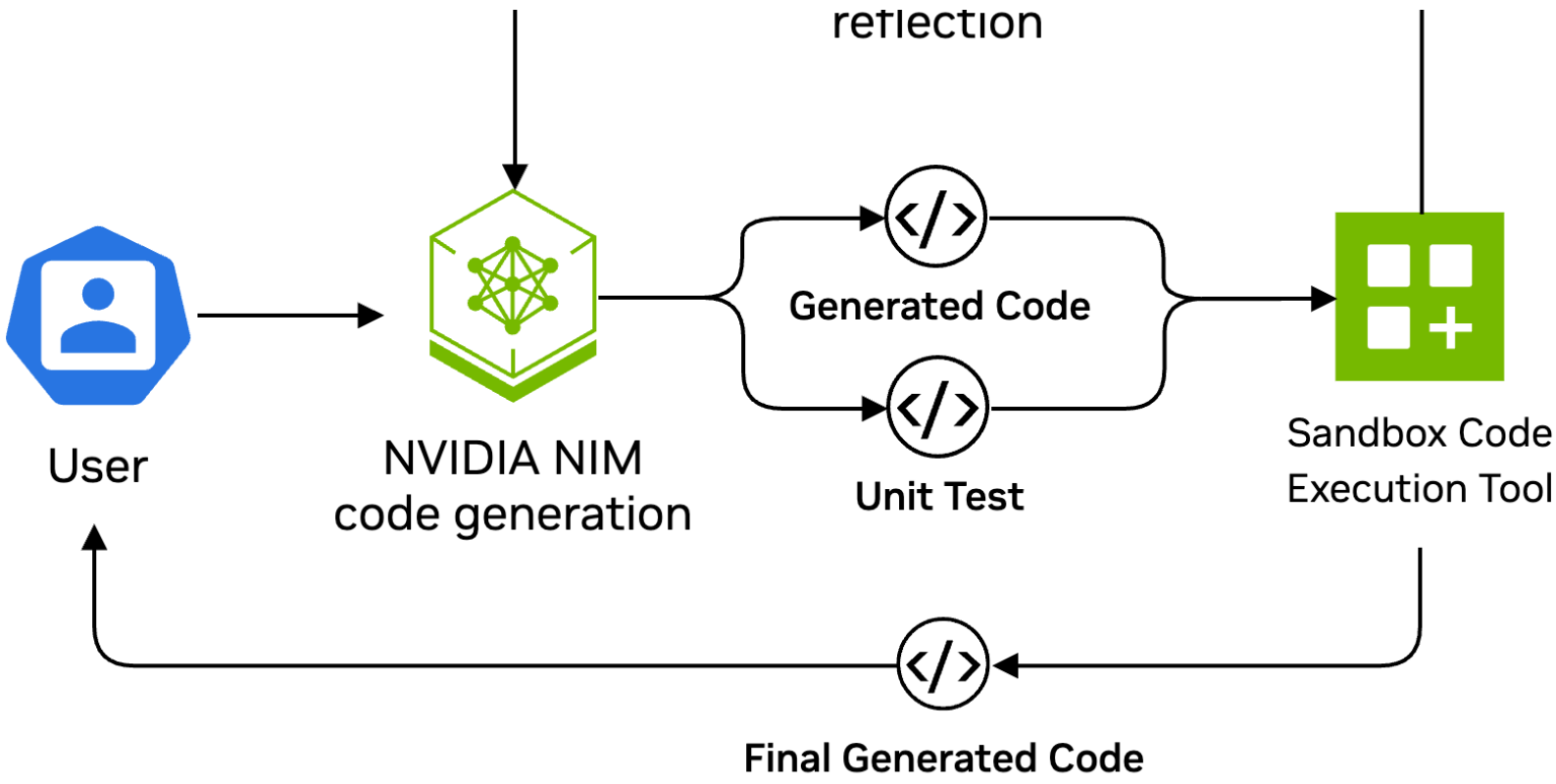


Figure 1. A code modification agent diagram

In this tutorial, we omitted some implementation details of the LangGraph pipeline. The [Agent toolkit examples directory]() contains various complete examples to get started.

Run the example workflow

Agent toolkit provides a CLI with various features including running a workflow, launching a server, and performing evaluations.

Run the workflow directly:

```
aiq run --config_file=examples/code_gen_example/configs/config.yml
```

```
--input 'Write a Python function named largest_rectangle that computes the
area of the largest rectangle in the histogram. Given an array heights of
non-negative integers representing the histogram bar heights where the
width of each bar is 1, return the area of the largest rectangle that can be
formed within the histogram. Use the following files:

test_path: "/home/aiq/rectangle_tests.py",

solution_path: “/home/aiq/rectangle_solution.py"'
```

The logs display in the console, and the agent can be easily integrated with the Agent toolkit user interface.

The following is an example of the output:

```
Configuration Summary:

--------------------

Workflow Type: react_agent

Number of Functions: 1

Number of LLMs: 3

Number of Embedders: 0

Number of Memory: 0

Number of Retrievers: 0

2025-02-27 17:33:27,459 - aiq.agent.react_agent.agent - INFO - The user's
question was: 'Write a Python function named largest_rectangle that
```

computes the area of the largest rectangle in the histogram. Given an array heights of non-negative integers representing the histogram bar heights where the width of each bar is 1, return the area of the largest rectangle that can be formed within the histogram. Use the following files:

test_path: "/home/aiq/rectangle_tests.py",

solution_path: “/home/aiq/rectangle_solution.py"'

2025-02-27 17:33:27,460 - aiq.agent.react_agent.agent - INFO - The agent's thoughts are:

Thought: To solve this problem, we need to write a Python function that calculates the area of the largest rectangle in a histogram.

Action: code_gen_tool

Action Input: {"problem_statement": "Write a Python function named largest_rectangle that computes the area of the largest rectangle in the histogram. Given an array heights of non-negative integers representing the histogram bar heights where the width of each bar is 1, return the area of the largest rectangle that can be formed within the histogram.", "solution_path": "/home/cmunley/aiq-225-2/rectangle_solution.py","test_path": "/home/cmunley/aiq-225-2/rectangle_tests.py"}

==========

STARTING NEW CODE GENERATION TASK

==========

Initial Code:

```
def largest_rectangle(heights):
```

Generating solution...

```
largest_rectangle(heights):
    stack = []
    max_area = 0
    index = 0
    while index < len(heights):
        if not stack or heights[index] >= heights[stack[-1]]:
            stack.append(index)
            index += 1
        else:
            top_of_stack = stack.pop()
            width = index if not stack else index - stack[-1] - 1
            area = heights[top_of_stack] * width
            max_area = max(max_area, area)
    while stack:
```

```
        top_of_stack = stack.pop()
        width = index if not stack else len(heights) - stack[-1] - 1
        area = heights[top_of_stack] * width
        max_area = max(max_area, area)
    return max_area
```

Test Results:

FAILED: Expected -1, got 0

PASS

PASS

Test Failed - Attempt 1/3

Analyzing errors:

The error is likely due to the fact that the function is not handling the case where the input list is empty. In this case, the function should return -1, but it's currently returning 0. [truncated for the sake of this post]

Generating updated solution...

```
largest_rectangle(heights):
    if not heights:
        return -1
    stack = []
    max_area = 0
    index = 0
    while index < len(heights):
        if not stack or heights[index] >= heights[stack[-1]]:
            stack.append(index)
            index += 1
        else:
            top_of_stack = stack.pop()
            width = index if not stack else index - stack[-1] - 1
            area = heights[top_of_stack] * width
            max_area = max(max_area, area)
    while stack:
        top_of_stack = stack.pop()
        width = index if not stack else len(heights) - stack[-1] - 1
```

```
        area = heights[top_of_stack] * width
        max_area = max(max_area, area)
    return max_area
```

```
Updated Test Results:
PASS
PASS
PASS
```

```
Tests passed successfully!
The agent's thoughts are:
Thought: The code generation tool has generated the Python function
largest_rectangle and the unit tests have passed, indicating that the
function is correct.
Final Answer: The final answer is that the Python function
largest_rectangle has been successfully generated and tested, and it
correctly calculates the area of the largest rectangle in a histogram.
```

Adding functions in configuration file to execute varied tasks

Adding capabilities to the supervisor agent, such as web search or calculator use, is as simple as adding

the functions in the configuration file. Agent toolkit provides many useful tools to get started. For more information and a full list of the tools available to agents by default, see the Agent toolkit tools folder.

Conclusion

Code generation problems are excellent candidates for test-time compute scaling because it's possible to identify when a solution is correct. For example, a test-driven development agent can iterate on proposed solutions, with the number of iterations limited only by a compute budget. Reasoning LLMs such as DeepSeek's R1 model provide reflections that can accurately guide a code generation model through a debugging process. Agentic tool use, memory, and planning can be integrated to improve the system.

The NVIDIA NeMo Agent toolkit library simplifies the development of agentic systems, providing reusable components and a simple toolkit compatible with the entire ecosystem and optimized for the best performance. By orchestrating different models, frameworks, and tools under a comprehensive and optimized toolkit, we're transforming the future of work by solving complex, real-world tasks.

For more information about how to use the Agent toolkit profiler. Sign up for the Agent Toolkit Hackathon and learn to build hands-on skills using the open-source toolkit that will help you advance your agentic systems.